

Participation Is Not Collaboration: Tracing Public Ownership After Cross-Product Agent Review

Dao Sy Duy Minh^{1*}, Huynh Trung Kiet¹, Tran Chi Nguyen¹,
Pham Phu Hoa¹, Nguyen Lam Phu Quy¹

^{1*}Affiliation pending, Institution pending, City pending, Country pending.

*Corresponding author(s). E-mail(s): [Corresponding email pending](#);

Abstract

Context: A pull request (PR) written by one coding-agent product and reviewed by another looks like a multi-agent workflow. Two products on one PR do not show that one addressed the other. **Objective:** We trace the public path from cross-product review to an addressed edge, a next owner, and a later state. **Method:** We screen 361,296 PRs in a pinned AIDev release and analyse 8,608 trigger PRs, 546 matched pairs, and a 1,067-PR outcome cohort. Exact account, comment, and review-batch identifiers anchor every event to one trigger. We set aside short automation bursts, match cross- to same-product triggers, trace earlier review history, and fix a check point at hour 48. **Results:** Direct dialogue between different products is rare. After a five-minute burst, user accounts own most remaining first actions, and most have reviewed in the repository before. Cross-product triggers get 13 percentage points less visible follow-up than matched same-product triggers. Among PRs still open at that point, an exact reply to the trigger is linked to 17 points more later merge. The link holds under stricter edge rules, a shuffle test inside each repository, and a whole-population model that drops the check point and lets the edge switch on in time. A user account writes that edge four times in five, so it marks human acknowledgement, not agent-to-agent dialogue. **Conclusions:** Public multi-agent impact should be reported as a map of connections, not as a count of products. Addressed edges and visible ownership are useful field signals, but they come from watching, not from an experiment.

Keywords: coding agents, multi-agent software engineering, code review, human-agent collaboration, pull requests, mining software repositories

047 1 Introduction

048
049 On a pull request today you can find two AI coding products at once. One writes the
050 change. A different one reviews it. It looks like two machines working as a team, and
051 it is easy to report it that way.

052 We followed what happens next, one event at a time. Almost none of it is teamwork.
053 The two products hardly ever answer each other. What we found instead was a person.
054 A maintainer reads the agent’s review point and replies to it. That reply is rare. It is
055 also the one public connection that goes together with the change being merged later.
056 A count of products on a pull request measures none of this.

057 The question is live. A recent position paper argues that agents can already meet
058 every goal of code review, and that keeping a person in the loop as the mandatory
059 reviewer “is a dead end” (Monperrus 2026). Our data speak directly to that: in this
060 corpus the person is not a formality on top of an agent loop, they are the only part of
061 the loop that visibly closes.

062 The gap also matters for measurement. A short burst of bot events can look like
063 a conversation. A new review can be part of the same API call. A user account can
064 act next without having been chosen as a go-between. Mining GitHub has long been
065 known to reward this kind of care (Kalliamvakou et al. 2016). If these cases are mixed
066 together, a product count becomes a weak measure of teamwork.

067 Field studies already tell us a lot. They count agent-authored PRs, measure how
068 common AI review is, model reviewer order, and study how developers answer agent
069 comments (Li et al. 2026; Selvanayagam and Ghaleb 2026; Zhong et al. 2026a; Cynthia
070 et al. 2026b; Khemissi et al. 2026; Duma et al. 2026). Several report that people
071 still carry the review (Zhong et al. 2026b; Chowdhury et al. 2026). Designed systems
072 show how known agents can pass messages, roles, files, or structured disagreement
073 (Destefanis and Aste 2026; Qiu and Gill 2026; Zhang 2026).

074 What is still missing is a hard test. These studies count who appears, or they read
075 what was written. Neither tells us whether one actor actually answered another. Our
076 question is narrower and stricter: after one cross-product review event, what connected
077 public path can we point at in the data?

078 We ask three questions:

079 *RQ1 – Is there a handoff?* Once we split one review batch into its parts, and set aside
080 the fast burst of events that follows a trigger, does anything left look like one product
081 handing over to another? And who takes the next visible step? The cost of hand-
082 ing a task between agents has been studied under controlled interruptions (KC and
083 Budathoki 2026). We ask the question that comes before it: can a public handoff be
084 seen at all?

085 *RQ2 – Who bridges the boundary?* How much public follow-up happens across the
086 boundary between two products? And when a person steps in, have they worked on
087 this repository before?

088 *RQ3 – Who writes the connection, and does it mark a later state?* Two signals say
089 that something connected: a reply pointing at the exact review comment, and automa-
090 tion followed by a person. Does either go together with the change being merged later,
091 and who is it that writes the reply?

092

RQ4 – What makes someone answer across the boundary? A pull request body often points at an issue. That pointer is written before any review, and it gives a reader context the diff alone does not. Does it change whether a review point gets answered, and does that depend on who is reviewing?

We make three contributions.

An evidence ladder with a hard rung. We separate four things that a product count runs together: both products appearing on one PR, a reply that points at the exact review comment, the next visible owner, and the later state of the PR. The second rung is machine-checkable, and it is strict. It is what makes the rest of the paper falsifiable.

A finding that the ladder produces. Each rung deletes most of what the rung above admitted. Almost no cross-product activity becomes one product answering another. The rare exact reply does mark a different later outcome, and a person writes it four times in five. So the measurement choice changes the conclusion, which is the point.

A reusable check for order-based claims. Reviewer order is often read as evidence of coordination. We show that shuffling the order of events inside the same PR reproduces the apparent pattern almost exactly. Prior work on review order compares against an alphabetical baseline (Bouraffa et al. 2025); we argue a shuffled baseline is the one that can falsify a coordination claim.

We do not claim the first observation of AI-to-AI review. Full data rules, all estimates, robustness checks, and rejected experiments are in Online Resource 1 (Supplementary Information).

2 Related Work

2.1 Agent work in public repositories

AIDev maps agent-authored work at GitHub scale (Li et al. 2026). Studies of this and related data examine acceptance, failure, developer roles, merge conflicts, review signals, and cross-PR references (Pinna et al. 2026; Ehsani et al. 2026; Cynthia et al. 2026a; Xu et al. 2026; Nachuma and Zibran 2026; Khemissi et al. 2026). These studies make clear that repository context and human participation matter. They also make merge a poor stand-alone measure of an agent’s quality.

Four lines of work sit closest to ours.

How common it is. The nearest study counts same-product and cross-product AI review, comment output, and speed of reply (Selvanayagam and Ghaleb 2026). It reports that cross-product review is rare and very fast. Our burst windows are a way of taking that speed seriously rather than a new discovery.

Who reviews, and in what order. Zhong et al. model ordered human, LLM, and agent reviewer sequences across review eras (Zhong et al. 2026a). Bouraffa et al. ask whether the order of review activity is meaningful at all, using an alphabetical baseline (Bouraffa et al. 2025). Neither compares the observed order against a shuffled one, which is the check we add.

139 *Who does the work.* Several studies find that people still carry the review. Zhong
140 et al. report that developers adopt human suggestions far more often than agent
141 suggestions (Zhong et al. 2026b). Chowdhury et al. report a lower merge rate for PRs
142 reviewed only by an agent than for PRs reviewed only by people (Chowdhury et al.
143 2026). Duma et al. report that most AI-generated PRs get no review at all (Duma
144 et al. 2026). Our RQ3 result agrees with the direction of this work. What we add is a
145 much stricter definition of what counts as the human contribution, and a design that
146 keeps it before the outcome.

147 *Whether agent feedback helps.* Cynthia et al. study how developers respond to
148 individual agent review comments (Cynthia et al. 2026b), and Lin et al. mine developer
149 feedback to a deployed agentic reviewer (Lin et al. 2026). Fatima et al. find that the
150 quality of reviewer-bot feedback shows no clear link to workflow outcomes (Fatima
151 et al. 2026). That null is worth holding next to our result: we do not measure whether
152 a review comment was good, only whether someone addressed it.

153 *Bots and review participation before agents.* The questions we ask are older than
154 coding agents. Review bots change how a PR behaves (Wessel et al. 2022, 2023).
155 Participation, rather than presence, is what relates to review outcomes (McIntosh
156 et al. 2016; Thongtanunam et al. 2017), and human factors decide whether an invited
157 reviewer engages at all (Ruangwan et al. 2019). First response time dominates how
158 long a PR takes (Zhang et al. 2022), which is why we place our check point after
159 it rather than before. Our contribution is to carry those questions across a product
160 boundary that did not exist when they were asked.

161 These papers occupy the broad claims that cross-product review exists, that
162 reviewer order matters, and that people answer agent feedback. Our unit is different.
163 It is one cross-product trigger, and the exact edges, batches, owners, and later state
164 that follow it.

165

166 2.2 Designed multi-agent review

167 Role-based systems such as ChatDev and MetaGPT make communication part of
168 the design (Qian et al. 2023; Hong et al. 2023). RepoReviewer divides review into
169 repository, file, priority, and summary roles (Zhang 2026). Adversarial Review adds
170 a critic that checks a review through structured disagreement (Qiu and Gill 2026).
171 SWE-Review tests a known generate-review-revise loop (Wang et al. 2026). Controlled
172 traces can therefore reveal roles and messages directly.

173 Public GitHub traces are weaker but more natural. Product co-presence does not
174 prove shared memory, a joint plan, or private communication. We use *cross-product*
175 *feedback* for the trigger and reserve *public coordination* for observable connections and
176 ownership. This measurement boundary is central to our novelty.

177

178 3 Data and Method

179

180 3.1 Data and event anchor

181 We pin AIDev-7.6M at revision 37bbe1533e26cc1e1374917dba1186d1c8a4dc81. The
182 release contains 7.69 million agent-authored PRs. We use the richer AIDev-pop layer:
183
184

361,296 PRs from repositories with more than 100 stars, a star-based filter of the kind used to separate engineered projects from scratch repositories (Munaiah et al. 2017). Its event tables contain submitted reviews, inline review comments, PR comments, and selected lifecycle events. PRs are created through 31 March 2026; we use 15 April 2026 at 00:00 UTC as the common observation boundary.

We map only exact official GitHub identities to the six AIDev product labels. Names that merely look similar are not guessed. The PR author carries one product label. A *trigger* is the first review, inline comment, or PR comment written by a different product. We take the earliest such event on the PR, and it must fall after the PR was opened and before it was closed. Everything we measure afterwards is measured from the trigger. Online Resource 1 gives the full identity registry, table coverage, joins, and denominators. Inline comments join to a submitted review by review ID and then to the PR.

An *addressed edge* means one thing only: a later comment carries the trigger comment’s own identifier as its reply target. A reply somewhere else on the same PR does not count. A later review must also carry a different review-batch ID. These rules stop one API call from appearing to answer itself.

We checked what that identifier actually points at, because the paper rests on it. GitHub sets the reply target to the *first* comment of an inline thread, not to whichever comment came just before. The release agrees: across 88,907 inline replies, not one names a comment that is itself a reply, so reply threads are one level deep. An addressed edge is therefore *a reply in the thread that the trigger opened*. Section 6 reports what that costs us, which turns out to be very little in this cohort. Figure 1 shows the anchor, the three exclusion rules, and the four evidence levels that the rest of the paper keeps separate.

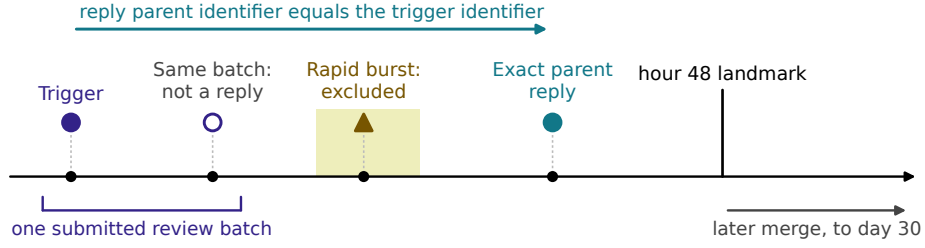
3.2 Three analytic views

A *relay* means the next visible step is taken by a product different from the one that wrote the trigger. It is the thing a multi-agent workflow would produce, and it is what RQ1 looks for.

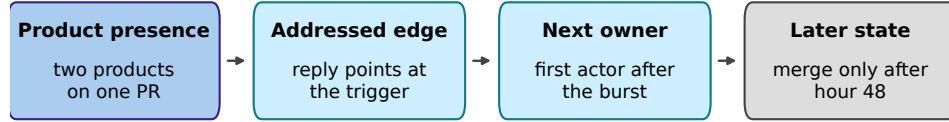
For RQ1, we follow each first cross-product trigger for seven days, ending at PR closure when earlier. We join exact replies, new review rounds, later PR comments, and visible force-push events. The first later owner is a user account, mapped product, other bot, branch movement or untyped activity, or no action. We repeat this state after excluding the first 0, 1, 5, 10, or 30 minutes. These windows test how much one fast automation burst is driving the answer. They do not reveal a private run boundary. We then apply a second five-minute waiting period, to test whether a mapped-product-first state really becomes a relay to another product.

For RQ2 we build matched pairs. Each cross-product trigger is paired with a same-product trigger from the same repository, the same PR-author account, the same author product, the same source channel, and the same month. Pairing uses nearest time without replacement. We also identify the first distinct user account within 48 hours. Prior public history requires that account to have submitted a review in the same repository on a different PR strictly before the trigger.

A One trigger, three exclusion rules



B Four evidence levels, read separately



A level never implies the level to its right. None of them observes a private run, a shared plan, or whether the review point was understood.

Fig. 1 How we measure. Panel A puts one cross-product trigger on a single PR and marks what each rule removes. A sibling comment in the same submitted review batch is not a reply. An event inside the tested burst window does not define the next owner. Only a comment whose parent identifier is the trigger's own identifier counts as an addressed edge. The later state is read after a fixed check point at hour 48. Panel B lists the four evidence levels in order. This is a drawing, not data: the positions are for illustration only.

For RQ4 the exposure is a pre-trigger property of the change: the PR body references an issue. The outcome is whether the review point gets answered within 48 hours. Two measurement choices matter and we enforce both. We rebuild the outcome from the raw comment table for cross-product and same-product triggers alike, because our cross-product event tables cannot supply the same-product side. And we restrict the comparison to triggers that open their own thread, because a mid-thread trigger cannot receive an edge at all: that affects 0.4% of cross-product triggers but 46% of same-product ones, so leaving it in would compare a possible outcome with an impossible one. We then compare the two boundaries inside the same repository and the same month.

For RQ3 we fix a check point at hour 48. We study only PRs that are still open at that point, and we count merges only after it. This keeps cause before effect in time. It is also a limit. Most cross-product inline-trigger PRs finish sooner, so they fall outside the group, and we report how many with the results rather than leaving it unsaid. The group also needs 30 complete days of later observation. The exposure is an exact addressed edge by hour 48; the outcome is merge after hour 48 and by day 30. A secondary cohort classifies the first 48 hours as user first, automation then user, automation without a user event, movement, other activity, or no action. This lets us compare an automation-to-user relay with automation alone.

We report percentage-point contrasts because they are easy to read. Main intervals cluster or resample by repository. Outcome models adjust for product, channel or month where relevant, trigger age, and public activity known before the trigger. We vary the addressed-edge window, add repository fixed effects, and remove each product pair in turn.

The later-state comparison comes from watching, not from an experiment. So we also ask how much hidden structure it would take to remove the result. We run four checks.

First, an E-value (VanderWeele and Ding 2017). It asks a simple question: how strong would a cause we did not measure have to be, before it could wipe out our result? A larger number means a more demanding requirement. Second, a grid that shows how big one missing yes-or-no factor would have to be. Third, three outcomes that had already finished before the trigger. The trigger cannot change the past, so these should show no difference; if one of them does, that is a warning sign and we report it as such. Fourth, a shuffle test that reassigns the exposure label inside each repository.

We also run two checks that attack the design rather than the arithmetic. The first drops the hour-48 rule and models the chance of merging over time across the whole trigger population, with the edge switching on when it actually happens. The second splits the edge by whether the person who wrote it had already reviewed in that repository, and then repeats that split inside single repositories.

These checks put a limit on fragility. They do not tell us what an intervention would do. Task difficulty and maintainer intent are not measured here. Either one could shape both the public path and the outcome.

3.3 Reproducibility and AI use

The artifact pins the data revision, stores PR-grain cohorts, generates all tables and figures, and runs automated data and leakage checks. We also screened current public review and agent-trace datasets against the same event contract. A released AI-to-AI cohort checks product-pair and event-time attribution on shared PRs. A full SWE-Review-Chat scan applies the exact identity, parent-edge, overlap, and landmark rules. External rows are not pooled with the main cohort. OpenAI Codex helped inspect data, write and test analysis code, search literature, make figures, and edit the draft. The authors checked source records, reran the analysis, verified cited sources, and reviewed the text and figures. Human authors remain responsible for every claim, citation, and final decision. No model is an author.

4 Results

4.1 RQ1: product presence rarely becomes a relay

Two products on one PR almost never becomes one product answering the other. Once the fast automatic burst is set aside, a person usually takes the next visible step.

Figure 2 gives two views of the same narrowing process, over the complete seven-day cohort. Panel A shows how many PRs remain as the connection rule gets stricter.

Panel B shows the first visible owner after each tested burst window, among PRs that still have an action. Later public activity is common. An exact reply from a product different from the triggering reviewer is not: it appears on fewer than 1% of PRs. User accounts write four in five of those exact replies.

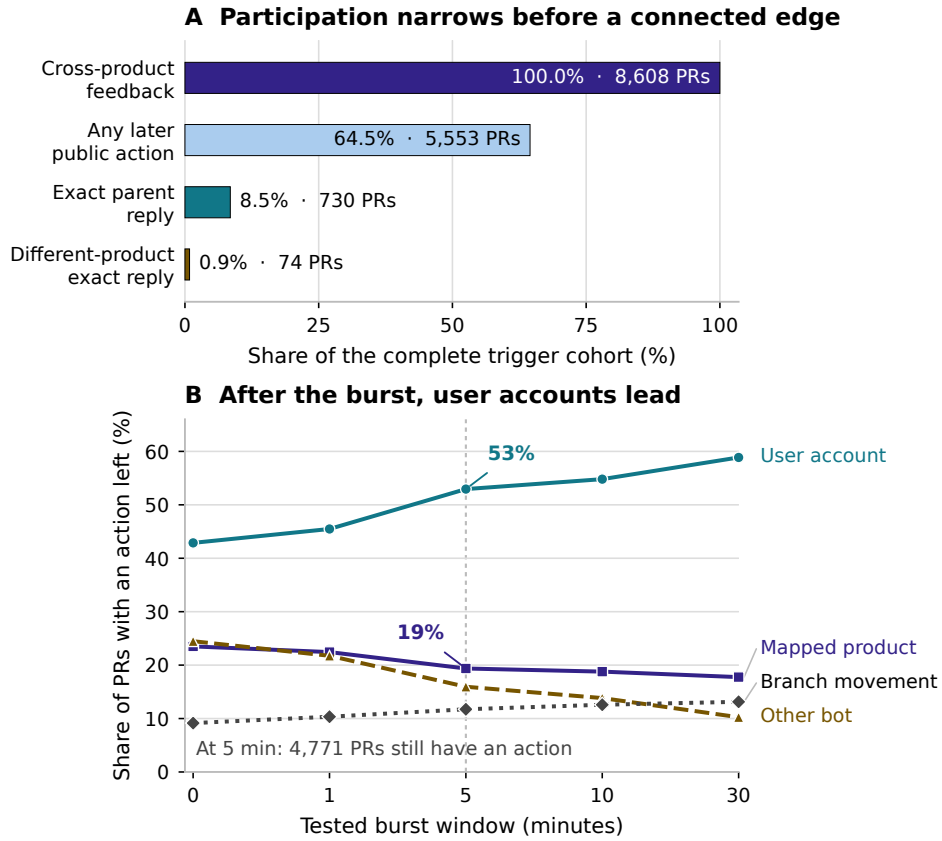


Fig. 2 Public activity narrows before any connected edge appears. Panel A shows the share and count of PRs that meet each stronger evidence rule; the full cohort holds 8,608 PRs. Panel B shows the first visible owner after each of five tested burst windows, among PRs that still have an action. At five minutes, 4,771 PRs still have one, and user accounts own 53% of those actions against 19% for mapped products. The windows are test settings, not known private run boundaries.

The burst test changes the owner, not only the time. After five minutes, mapped products lose close to a third of their mapped-first classifications. User accounts then own far more of the remaining actions than mapped products do. This ordering holds in every leave-one-repository-out and leave-one-product-pair-out check.

The next step is even more selective. Among PRs that are still mapped-product first after five minutes, only about one in twenty move next to a different mapped product after a second waiting period.

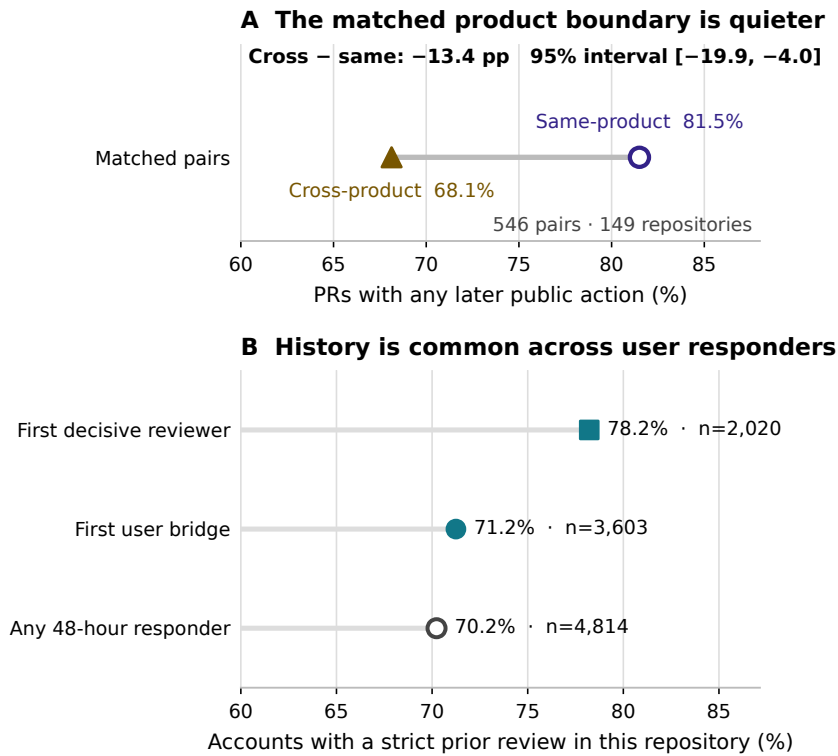


Fig. 3 A quieter boundary, and a history that is common on both sides of it. Panel A compares later public action for cross- and same-product triggers in nearest-time matched pairs. Pairs share repository, author account, author product, source, and month. The sample holds 546 pairs from 149 repositories, with follow-up after 68.1% of cross-product and 81.5% of same-product triggers; the gap is 13.4 points, 95% interval -19.9 to -4.0 from resampling repositories. Panel B compares the first user bridge, the first later decisive reviewer, and all users who answered within 48 hours. A decisive reviewer is one who approves or asks for changes. History means a public review on another PR in the same repository before the trigger. The open circle marks the wider responder baseline. A past review does not tell us whether an account used a tool, remembered the repository, worked by hand, or was picked on purpose.

The same product carrying on is far more common, and it is tempting to read that as an agent staying with the task. It is not. We shuffled the order of the later events inside each PR, keeping that PR's own mix of products, and the shuffled data reproduce the pattern almost exactly. So the trace supports one thing only: a relay to another product is rare. It gives no support for a special product-persistence mechanism.

Answer to RQ1. Two products on one PR greatly overstates how much handing over really happens. Once the fast burst of events is set aside, user accounts are the main visible owner. A relay to another mapped product is rare.

4.2 RQ2: familiar user accounts bridge a quiet boundary

The cross-product boundary is a quieter one. Comparable triggers get less visible follow-up across it, and when someone does step in, it is usually a person who has reviewed in that repository before.

Visible follow-up is less common after a cross-product trigger than after its matched same-product partner. The gap is 13.4 percentage points (Figure 3A). No other outcome supports a claim that this quieter boundary leads to more branch movement or more merges.

Figure 3 joins the matched comparison with a wider history check. Seven in ten of the first user-account bridges had already reviewed another PR in that repository. That rate is close to the rate among all users who answered within 48 hours. So prior history is common right across the response layer. It does not tell us why one account rather than another becomes the bridge.

Answer to RQ2. Comparable cross-product feedback has less public follow-up. When a user account becomes the visible bridge, it is usually not a repository newcomer, but the trace does not show deliberate expert routing.

4.3 RQ3: a person writes the edge that marks a later state

Among the slower PRs that are still open at hour 48, one signal stands out as a marker of later merge: an exact reply to the trigger. Four times in five, a person writes it.

The group holds 1,067 inline-trigger PRs. It is the slower-moving part of the population. Most cross-product inline-trigger PRs close, and most of those merge, at or before hour 48. Our group is the 27% that are still open when the outcome window opens. Everything below describes that slower group, not the most common kind of cross-product PR.

The exposure also needs naming precisely. The rule is a reply whose parent identifier is the trigger's own identifier. It does not require another product to write that reply. Across the exposed reply events, user accounts write the large majority. The triggering product replies to its own comment in a small number of cases, a few more come from bots we do not map to a product, and only three come from a mapped product other than the one that wrote the trigger. So the connected signal here is a person acknowledging an agent's review point. It is not a conversation between agents.

Later merge is more common when there is an exact reply to the trigger than when there is none (Figure 4A). After adjusting for what was already visible before the trigger, the difference is 17.3 percentage points. It stays positive for reply windows from one to 48 hours, in a model that only compares PRs inside the same repository, and after removing each ordered product pair in turn. We also tested it against a harder comparison. Instead of an edge versus silence, we compared an exact edge with PRs that already carry other public discussion. The result stays positive.

Because most exposed replies are written by people, we refit the model under two stricter rules for what counts as an edge. One drops replies written by the triggering product itself. The other keeps only replies from a user account. Both give the same answer, with intervals that stay clear of zero (Figure 4B, rows two and three). The result is a property of the addressed edge, not of a product answering itself. Only three

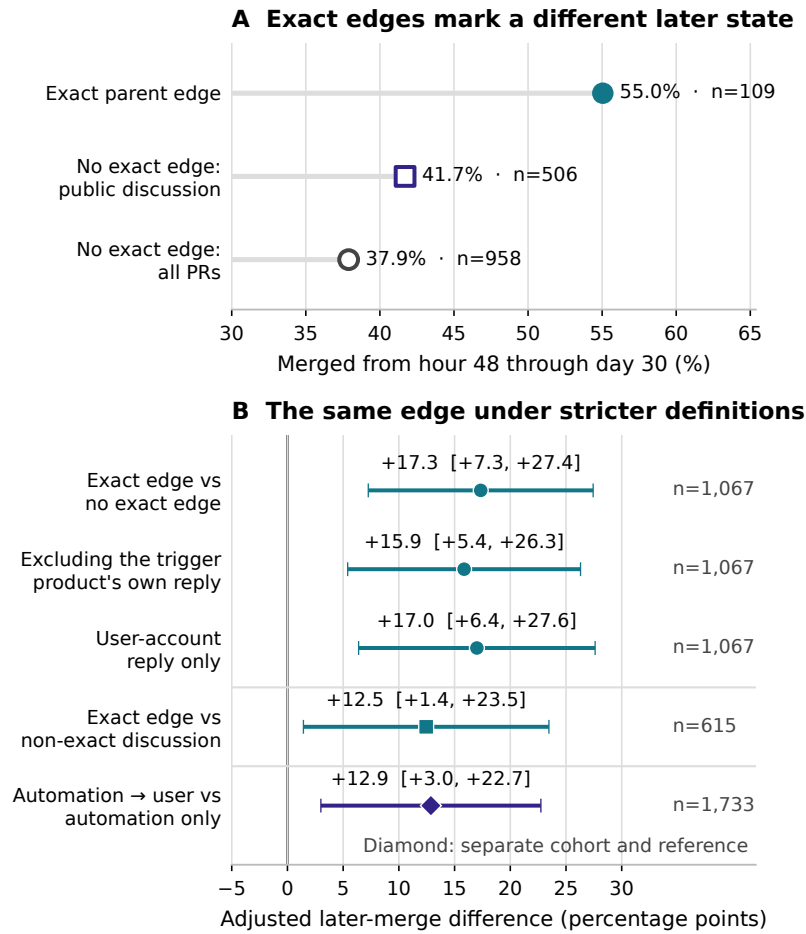


Fig. 4 Connected public signals and later merge. Panel A shows raw later-merge rates: 55% with an exact edge, 41.7% for other public discussion without one, and 37.9% across all PRs without one. The discussion group sits inside the no-edge group. Panel B shows adjusted differences. The top three rows are the same group of PRs under three rules for what counts as an edge: any exact parent reply, the same edge after dropping replies written by the triggering product itself, and edges written by a user account. The fourth row is the discussion-only control. The fifth, marked with a diamond, is the separate automation-to-user route, which has its own group and its own reference. Each row prints its estimate and 95% interval. These are links in observed data, not causes.

PRs carry an exact reply from a mapped product other than the trigger's. That is far too few to model. The count is the RQ1 finding showing up again at the outcome stage.

The exact edge is not the only visible thing that goes with a later merge. A wider model that puts all four post-trigger states side by side shows a small group of PRs with branch movement and no discussion sitting even higher, on a very wide interval. Any visible sign that a PR is still being worked on points the same way. We report

that model in Online Resource 1 and keep the harder discussion-only comparison as our main specificity check.

The automation-to-user route gives a separate check on a wider group. It is linked to 12.9 points more later merge than automation alone, after the same kind of pre-trigger adjustment. Figure 4 first compares raw later-merge rates, including other public discussion without an exact edge. Panel B shows two exact-edge comparisons and the separate automation-to-user check. The diamond marks the row with a different cohort and reference.

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

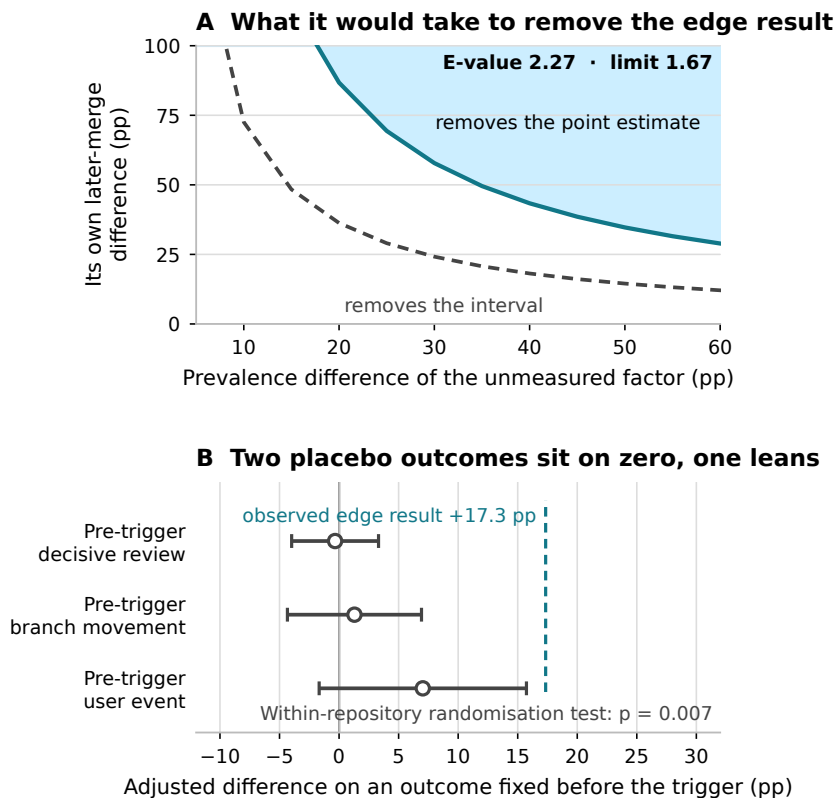


Fig. 5 How much hidden structure it would take to remove the exact-edge result. Panel A shows the arithmetic directly. One missing yes-or-no factor, fixed before the trigger, removes the result only inside the shaded corner. To get there it must be much more common among edge PRs *and* carry a large later-merge difference of its own. The dashed line is the weaker requirement that removes only the interval. As a single summary, the E-value is 2.27, and 1.67 at the interval limit. Panel B applies the same adjusted model to three outcomes that had already finished before the trigger, so the trigger cannot produce them. The teal line marks the observed edge result for scale. Two of the three sit on zero. Earlier user activity leans positive here and clears zero at a one-hour reply window, so the text reports its estimate rather than treating it as a pass. These are limits on fragility, not evidence of cause.

Figure 5 asks how fragile that result is. The E-value here is 2.27. A hidden cause would have to be tied to both the exact edge and to later merge by at least that much, on top of every control we already measure, before it could explain the result away. Panel A shows the same idea as a grid. The shaded corner is where such a factor would have to sit, and it is a demanding corner.

A shuffle test gives a second check, one that rests on the design rather than on a formula. We keep only repositories that hold both kinds of PR. Inside each one we shuffle the exposure label many times and refit. The shuffled results centre on zero, and the real result sits far out in the tail ($p = 0.007$, two-sided). So the finding does not rest on the standard interval formula.

The three before-the-fact outcomes are the weakest of these checks, so we report them as estimates rather than as a verdict. Two of them, an earlier decisive review and earlier branch movement, sit close to zero at every reply window. The third does not. Earlier user activity is +7.0 points at the 48-hour window, with a 95% interval of -1.7 to 15.7 . At one hour it is +16.7 points, and that interval clears zero. The one-hour value is a plain failure, and it is why 48 hours is our main window. The 48-hour value is a small leftover, based on too few PRs to be sure. It points in exactly the direction a doubtful reader would expect, so we report it as an estimate and not as a clean zero.

Two further checks matter more than any of the bounds above, because they attack the design rather than the arithmetic.

The first drops the hour-48 rule. Instead of keeping only PRs that are still open, we follow every cross-product inline-trigger PR from its trigger and ask how the chance of merging in the next stretch of time changes once an exact reply exists. A PR counts as unexposed until its own reply arrives. The odds of merging are about 1.7 times higher while an edge is active (Figure 6A), and the value barely moves as controls are added. So the result is not an artefact of studying only slower PRs.

The second asks who wrote the edge. We split the exposed PRs by whether the replier had already reviewed in that repository before the trigger. Almost all of the association sits with the familiar repliers. A reply from someone new to the repository shows no positive link at all (Figure 6B). That looks like a mechanism, and we were tempted to report it as one. It does not survive the obvious check. When the comparison is held inside a single repository, the gap between the two kinds of replier shrinks to near zero with an interval that spans it. Familiar repliers and newcomers are mostly in *different repositories*, and those repositories differ in how often anything gets merged. We therefore report the split as a description of where the signal sits, not as evidence that knowing the repository is what makes the reply work.

An exact reply may mark a clearer issue, an engaged maintainer, a useful comment, or another unmeasured condition. The data do not tell us which. The result supports the value of a connected trace as a field signal; it does not show that adding a reply would cause a merge.

Answer to RQ3. A person writes the connection, and the connection marks a later state. The exact addressed edge survives every check we ran, including dropping the hour-48 rule and letting the edge switch on in time. Four times in five it is written by a user account rather than by an agent. Nearly all of its association sits with repliers who already review in that repository, but that split is between repositories rather

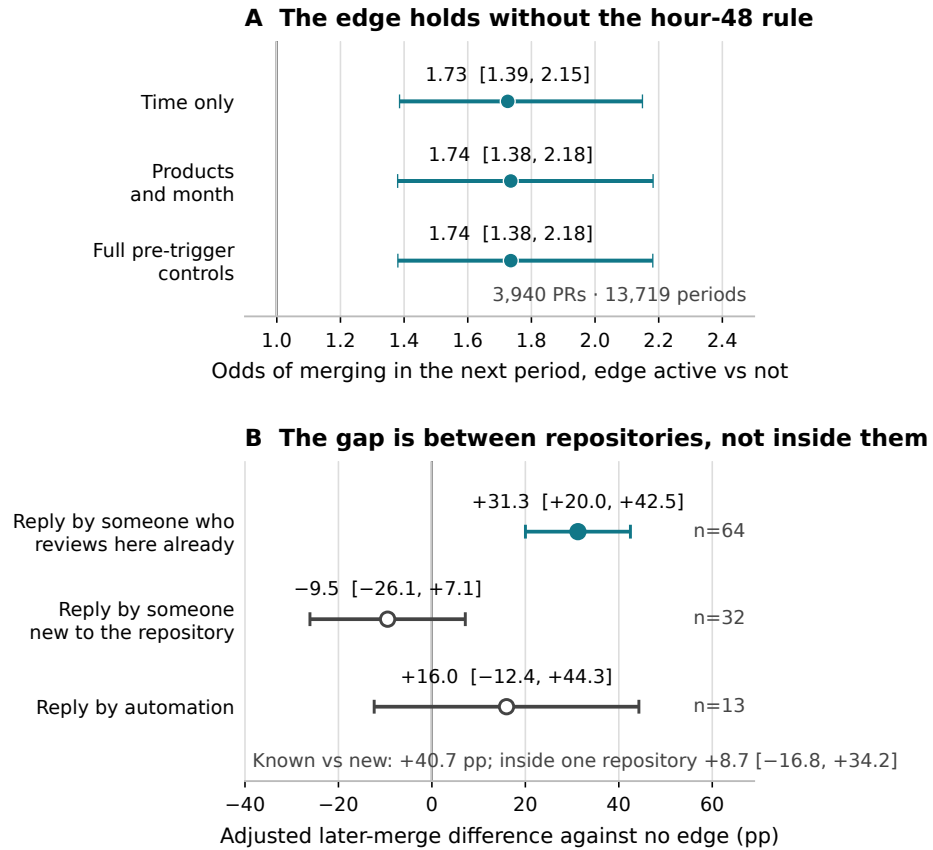


Fig. 6 Two harder tests of the same result. Panel A drops the hour-48 rule entirely. It follows every cross-product inline-trigger PR from its trigger and asks how the chance of merging in the next stretch of time changes once an exact reply exists. A PR counts as unexposed until its own reply arrives, so no time before the reply is credited to it. Panel B splits the edge by who wrote it, against PRs with no edge. The gap between a familiar replier and a newcomer is large, but it nearly disappears when the comparison is held inside one repository, so the trace cannot separate the person from the project.

than inside them, so we cannot show that familiarity is the reason. Nothing here is causal.

4.4 RQ4: task context matters only across the boundary

An issue link changes nothing when the reviewer is the same product. It roughly doubles the chance of an answer when the reviewer is a different one.

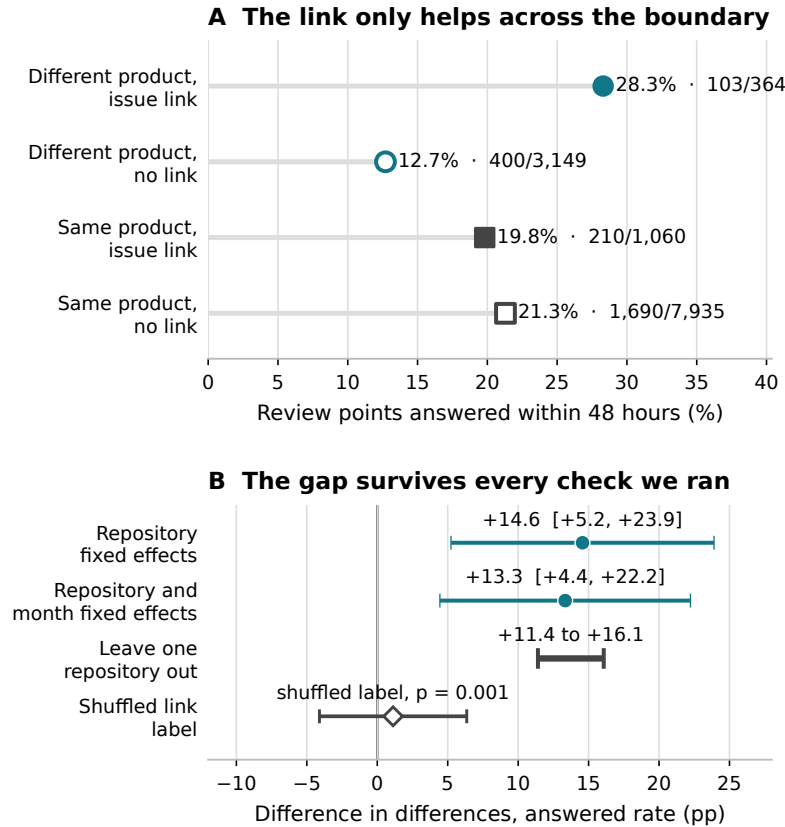


Fig. 7 An issue link in the PR body and whether the review point gets answered, among triggers that open their own thread. Panel A shows the four cells. Within one product the link does nothing. Across a product boundary the answered rate more than doubles. Panel B shows the difference between those two differences, under repository fixed effects and then with month added, and the range left by dropping each of the fifteen repositories that contribute most linked cross-product PRs. The dashed marker is the same contrast when the link label is shuffled inside each repository. These are links in observed data, not causes.

Among triggers that open their own thread, a same-product review point is answered about a fifth of the time whether or not the PR points to an issue. A cross-product review point is answered 12.7% of the time without a link and 28.3% of the time with one (Figure 7A). The difference between those two differences is 13.3 percentage points (95% interval: 4.4 to 22.2) once we compare inside the same repository and the same month.

691 We tried to break it. The estimate holds when the month is dropped, when
692 mid-thread triggers are put back in, and when each of the fifteen most influential
693 repositories is removed in turn, which leaves a range of 11.4 to 16.1 points. Shuffling
694 the link label inside each repository, the way we shuffle event order in RQ1, gives a
695 reference distribution centred on zero against which the observed value sits far out in
696 the tail ($p = 0.001$).

697 The direction is not new on its own. Issue linkage is already known to relate to
698 how agent-authored PRs fare, while showing no such relation for human-authored ones
699 (Yoshioka et al. 2026). What is new here is that the link is not really a property of
700 the pull request. It is a property of the boundary being crossed. The same link, on the
701 same kind of PR, does nothing when the reviewer shares the author’s product and a
702 great deal when it does not.

703 That fits what is known about how people read a review. References in a PR dis-
704 cussion are how participants build common ground (Chopra et al. 2021), and reviewers
705 begin by orienting themselves to context and rationale before they decide anything
706 (Gullstrand Heander et al. 2026). Inside one product the loop can close without that
707 work. Across the boundary somebody has to reconstruct why the change exists, and
708 a link to the issue is the cheapest way we can observe of handing them that.

709 **Answer to RQ4.** Pre-existing task context is what precedes an answer across the
710 product boundary, and only across it. We cannot say the link causes the answer. PR
711 bodies can be edited, and the release carries no timestamp for the link, so we cannot
712 prove every link was present before the trigger.

713

714 5 Discussion

715

716 5.1 What this changes

717

718 The picture a product count paints is wrong in a specific way. It shows two agents in
719 conversation. The trace shows something else: a thin map in which the agents rarely
720 speak to each other, and in which the connection that matters is a person answering
721 an agent. That person is usually not new to the repository. Their reply is rare, it is
722 easy to record, and it is the one public signal in our data that goes together with the
723 change being merged later. This is a link we observed, not a cause we proved. But it
724 points at a different thing to build and a different thing to measure.

725 We also tried, and failed, to name the mechanism. Splitting the edge by whether
726 the replier already reviews in that repository produces a very large gap. Held inside one
727 repository, the gap goes away. The honest reading is that our signal is a composite: it
728 marks a project where people answer review points and PRs get merged, and the trace
729 cannot pull the person and the project apart. That is a limit worth stating plainly,
730 because a reader who saw only the split would draw a stronger conclusion than the
731 data support.

732 Prior work has shown that several reviewer types can appear in one PR, and that
733 their order can relate to review outcomes. Our narrower result explains what that
734 order hides. One review batch can create several events. A short burst can look like
735 a handoff. A product that seems to keep going may just reflect which products are

736

installed. And once a person takes over, the work can bounce back to either product. Exact identifiers let us tell these paths apart.

The main field pattern is therefore not a self-running team. It is a thin map of who takes over next. Different-product dialogue is rare, user accounts often bridge the visible boundary, and the most concrete edge is linked to a different later state. This event-level view links natural repository data to the explicit message graphs used in designed multi-agent systems.

5.2 Implications for agentic software engineering

Record the edge, not only the actors. Review platforms and the platforms that run agents should store four things: what a message replies to, the review-batch ID, the run or session ID, and who is expected to act next. A timestamp on its own cannot separate a burst of parallel events from a real handoff.

Make ownership transfer visible. A product should be able to mark a review point as received, declined, routed, or awaiting a user. This would turn an inferred event sequence into an auditable coordination contract. The rare different-product relay shows that co-installation alone does not provide this contract.

Use connected signals to guide tests, not as success scores. The exact edge is a promising marker for checking work, because it is clear and stable. But it is rare. Only 8.5% of trigger PRs carry one, and about one in ten PRs in the hour-48 group do. It rarely fires by mistake, which makes it useful for auditing. It is not a score, and not yet a quality metric. A prospective study should randomize an explicit acknowledgement or routing feature and then measure resolution, rework, review time, and merge.

Support repository-aware human bridges carefully. Visible user bridges often have prior review history, so systems can offer context from past reviews when asking for help. However, the high baseline among all responders means history should not be sold as proof of expert selection.

5.3 Trying to prove ourselves wrong shaped the story

Several attractive claims did not survive stronger checks. A product that appeared to keep going was matched by a random-order shuffle of the same PR's own events. The newest example is the one above: repository familiarity looked like the reason the edge works, until the comparison was held inside a single repository. Repeated automation did not show a special escalation dose. Post-burst ownership moved in both directions rather than showing one-way user takeover. Prior review history did not support a stable within-repository merge advantage. Request events lacked the target identity needed to prove deliberate routing. Structural comment overlap also remains semantically uncoded. We keep these results and all gates in Online Resource 1 rather than turning them into positive headlines. We also ran the same rules over a large review dataset packaged by other people. After removing the PRs that overlap ours, it left no separate group of exact-edge PRs at the hour-48 check point. That is a failed replication gate, not proof that such edges never occur. It shows why being in the same thread cannot replace a reply that points at the exact comment.

783 6 Threats to Validity

784
785 **What we can see.** The event tables do not cover the same PRs. A missing row can
786 mean there was no event, or that the event was not captured. Ordinary commit and
787 review timeline rows carry no usable time in this release. So we make claims about
788 timing only for event types that do carry a usable time. We also treat a force-push as
789 branch movement, not as a checked fix.

790 **Identity and meaning.** Our list of approved accounts is strict, so it may miss
791 some agent accounts. A GitHub `User` account does not prove that a person worked
792 alone. An exact parent link proves that someone addressed the point. It does not
793 prove understanding, agreement, or repair. We never see private messages, prompts,
794 or shared memory.

795 **Which PRs we kept.** The hour-48 check point keeps only PRs that are still
796 open at that moment, and most cross-product inline-trigger PRs close before then.
797 The landmark numbers therefore describe slower PRs. The whole-population model
798 in Figure 6A is our answer to this: it uses every trigger PR, and it still finds the same
799 direction. Shorter reply windows raise a further problem, because a reply inside such
800 a window could itself change whether the PR is still open at hour 48. We treat those
801 windows as extra checks only, and the 48-hour window is the main one.

802 **What the edge means.** An exact parent edge records that someone addressed
803 the trigger. It does not record who. In this group that someone is almost always a user
804 account, so RQ3 measures a person acknowledging an agent, not one product handing
805 over to another.

806 The reply identifier is looser than the name suggests. It points at the first comment
807 of a thread, so our edge means “a reply in the thread the trigger opened”. Two things
808 follow, and we measured both. First, the exposure can only fire when the trigger
809 is itself the first comment of its thread. Among cross-product inline triggers that is
810 almost always true: 13 of 3,526 sit mid-thread, and in the hour-48 group only 3 of
811 1,067 do, none of them exposed. The restriction bites hard for same-product triggers,
812 where 46% sit mid-thread, which is one more reason we do not read the same-product
813 edge rate as comparable. Second, inside a thread with several replies, a later reply may
814 be answering an earlier reply rather than the trigger. Among the 109 exposed PRs,
815 92 have exactly one reply in the thread, so for those the reply can only be answering
816 the trigger. For the remaining 17 we cannot tell.

817 **Selection and cause.** Matching holds several visible factors fixed, and the out-
818 come models use pre-trigger controls and repository-aware intervals. They cannot
819 balance what a review meant, how hard the task was, what the maintainer intended,
820 or what happened in private.

821 The stress test puts a limit on this. A hidden cause weaker than an E-value of 2.27
822 cannot account for the edge result. A stronger one still could. And the before-the-
823 fact user-activity check shows that selection of this kind is present in our data, and
824 only partly measured. Our controls also include no stand-in for task difficulty. The
825 later-state findings stay associations.

826 **How far the results reach.** AIDev-pop covers public repositories with more
827 than 100 stars, and the product pairs are uneven. Leave-one-out checks ease the worry
828 that a few repositories or a few pairs drive the result. They do not extend it to private

projects or to future versions of these products. The outside dataset we screened checks our schema and identities, but after removing overlap it gives no independent group for RQ3.

7 Conclusion

Two AI products on one pull request is not proof of teamwork. Once the fast burst of events is set aside, product co-presence almost never becomes one product answering another. User accounts form the main visible bridge, and most of them have reviewed in that repository before.

The clearest connected signal is an exact reply to the trigger. It marks more later merges, and it survives stricter edge rules, a shuffle test, and a model that drops the hour-48 rule and uses the whole population. A person writes that signal four times in five. So the edge that carries the weight is a human answering an agent, and that is exactly what a count of products does not measure. Field evaluation of multi-agent work should ask for three things: an addressed edge, a visible owner, and a later state.

Statements and Declarations

Funding: Pending author confirmation. **Competing interests:** Pending author confirmation. **Data availability:** The source data are the public AIDev-7.6M release at the pinned revision reported in Section 3. The derived cohorts needed to reproduce the results will be archived at [ARTIFACT DOI PENDING]. **Code availability:** Analysis and figure code will be archived with the derived cohorts at [ARTIFACT DOI PENDING]. **Ethics approval:** Final institutional wording is pending author confirmation for this public repository trace study. **Consent to participate:** Pending author confirmation. **Consent for publication:** Pending author confirmation. **Author contributions:** Pending author confirmation.

References

- Bouraffa A, Brandt C, Zaidman A, et al (2025) Not one to rule them all: Mining meaningful code review orders from GitHub. In: Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering
- Chopra A, Mo R, Dodson S, et al (2021) “@alex, this fixes #9”: analysis of referencing patterns in pull request discussions. Proceedings of the ACM on Human-Computer Interaction 5(CSCW2):385. <https://doi.org/10.1145/3479529>
- Chowdhury K, Banik D, Ferdous KM, et al (2026) From industry claims to empirical reality: An empirical study of code review agents in pull requests. In: Proceedings of the 23rd International Conference on Mining Software Repositories
- Cynthia ST, Das JK, Roy B (2026a) Are we all using agents the same way? an empirical study of core and peripheral developers’ use of coding agents. In: Proceedings of the 23rd International Conference on Mining Software Repositories, pp 520–532, <https://doi.org/10.1145/3793302.3793377>

875 Cynthia ST, Widyasari R, Roy B, et al (2026b) “Go Home Copilot, You’re Drunk”:
876 Understanding developer responses to agent-generated code review comments. arXiv
877 preprint arXiv:260721997 <https://doi.org/10.48550/arXiv.2607.21997>

878 Destefanis G, Aste T (2026) When agents coordinate: Measuring coordination in multi-
879 agent AI coding. arXiv preprint arXiv:260816801 [https://doi.org/10.48550/arXiv.](https://doi.org/10.48550/arXiv.2608.16801)
880 [2608.16801](https://doi.org/10.48550/arXiv.2608.16801), URL <https://arxiv.org/abs/2608.16801>

881 Duma K, Wróblewski P, Bobińska J, et al (2026) These aren’t the reviews you’re look-
882 ing for: How humans review AI-generated pull requests. In: Proceedings of the 30th
883 International Conference on Evaluation and Assessment in Software Engineering

884 Ehsani R, Pathak S, Rawal S, et al (2026) Where do AI coding agents fail? an empirical
885 study of failed agentic pull requests in GitHub. arXiv preprint arXiv:260115195
886 <https://doi.org/10.48550/arXiv.2601.15195>

887 Fatima SK, Abrar Y, Tahir AR, et al (2026) On the footprints of reviewer bots’
888 feedback on agentic pull requests in OSS GitHub repositories. In: Proceedings of
889 the 23rd International Conference on Mining Software Repositories

890 Gullstrand Heander L, Söderberg E, Rydenfält C (2026) Code review as decision-
891 making: building a cognitive model from the questions asked during code review.
892 Empirical Software Engineering 31(3):54. <https://doi.org/10.1007/s10664-025-107>
893 [91-2](https://doi.org/10.1007/s10664-025-107)

894 Hong S, Zhuge M, Chen J, et al (2023) MetaGPT: Meta programming for a multi-
895 agent collaborative framework. arXiv preprint arXiv:230800352 <https://doi.org/10>
896 [.48550/arXiv.2308.00352](https://doi.org/10.48550/arXiv.2308.00352)

897 Kalliamvakou E, Gousios G, Blincoe K, et al (2016) An in-depth study of the promises
898 and perils of mining GitHub. Empirical Software Engineering 21(5):2035–2071. [ht](https://doi.org/10.1007/s10664-015-9393-5)
899 [tps://doi.org/10.1007/s10664-015-9393-5](https://doi.org/10.1007/s10664-015-9393-5)

900 KC D, Budathoki A (2026) Handoff debt: The rediscovery cost when coding agents
901 take over interrupted tasks. arXiv preprint arXiv:260602875 <https://doi.org/10.485>
902 [50/arXiv.2606.02875](https://doi.org/10.48550/arXiv.2606.02875)

903 Khemissi I, Chouchen M, Wang D, et al (2026) Humans integrate, agents fix:
904 How agent-authored pull requests are referenced in practice. arXiv preprint
905 arXiv:260404059 <https://doi.org/10.48550/arXiv.2604.04059>, URL [https:](https://arxiv.org/abs/2604.04059)
906 [/arxiv.org/abs/2604.04059](https://arxiv.org/abs/2604.04059), accepted to the MSR 2026 Mining Challenge

907 Li H, Zhang H, Hassan AE (2026) AIDev: Studying AI coding agents on GitHub. In:
908 Proceedings of the 23rd International Conference on Mining Software Repositories,
909 pp 1029–1033, <https://doi.org/10.1145/3793302.3797249>

910 Lin HY, Liang M, Thongtanunam P, et al (2026) Is agentic code review helpful?
911 mining developers’ feedback to CodeRabbit reviews in the wild. arXiv preprint
912 arXiv:260703316 <https://doi.org/10.48550/arXiv.2607.03316>

913 McIntosh S, Kamei Y, Adams B, et al (2016) An empirical study of the impact of
914 modern code review practices on software quality. Empirical Software Engineering
915 21(5):2146–2189. <https://doi.org/10.1007/s10664-015-9381-9>

916 Monperrus M (2026) The end of code review: Coding agents supersede human
917 inspection. arXiv preprint arXiv:260613175

918
919
920

Munaiah N, Kroh S, Cabrey C, et al (2017) Curating GitHub for engineered software projects. <i>Empirical Software Engineering</i> 22(6):3219–3253. https://doi.org/10.1007/s10664-017-9512-6	921 922 923
Nachuma C, Zibran M (2026) When AI teammates meet code review: Collaboration signals shaping the integration of agent-authored pull requests. arXiv preprint arXiv:260219441 https://doi.org/10.48550/arXiv.2602.19441	924 925 926
Pinna G, Gong J, Williams D, et al (2026) Comparing AI coding agents: A task-stratified analysis of pull request acceptance. arXiv preprint arXiv:260208915 https://doi.org/10.48550/arXiv.2602.08915	927 928 929
Qian C, Liu W, Liu H, et al (2023) ChatDev: Communicative agents for software development. arXiv preprint arXiv:230707924 https://doi.org/10.48550/arXiv.2307.07924	930 931 932
Qiu ES, Gill J (2026) Adversarial review: Structured disagreement for grounded agentic code review. arXiv preprint arXiv:260818167 https://doi.org/10.48550/arXiv.2608.18167 , URL https://arxiv.org/abs/2608.18167 , accepted to the ICML 2026 Workshop on DL4C	933 934 935 936
Ruangwan S, Thongtanunam P, Ihara A, et al (2019) The impact of human factors on the participation decision of reviewers in modern code review. <i>Empirical Software Engineering</i> 24(2):973–1016. https://doi.org/10.1007/s10664-018-9646-1	937 938 939 940
Selvanayagam N, Ghaleb TA (2026) AI-to-AI code reviews of GitHub pull requests. In: <i>Proceedings of the 20th International Symposium on Empirical Software Engineering and Measurement</i> , https://doi.org/10.48550/arXiv.2608.21311 , emerging Results, Vision, and Reflection Papers Track; arXiv:2608.21311	941 942 943 944
Thongtanunam P, McIntosh S, Hassan AE, et al (2017) Review participation in modern code review. <i>Empirical Software Engineering</i> 22(2):768–817. https://doi.org/10.1007/s10664-016-9452-6	945 946 947
VanderWeele TJ, Ding P (2017) Sensitivity analysis in observational research: Introducing the E-value. <i>Annals of Internal Medicine</i> 167(4):268–274	948 949 950
Wang R, Chen J, Wang S, et al (2026) SWE-Review: Closing the loop on issue resolution with agentic code review. arXiv preprint arXiv:260706065 https://doi.org/10.48550/arXiv.2607.06065 , URL https://arxiv.org/abs/2607.06065	951 952 953
Wessel M, Serebrenik A, Wiese I, et al (2022) Quality gatekeepers: investigating the effects of code review bots on pull request activities. <i>Empirical Software Engineering</i> 27(5):108. https://doi.org/10.1007/s10664-022-10130-9	954 955 956
Wessel M, Vargovich J, Gerosa MA, et al (2023) GitHub Actions: the impact on the pull request process. <i>Empirical Software Engineering</i> 28(6):131. https://doi.org/10.1007/s10664-023-10369-w	957 958 959
Xu G, Subramanian A, Karthik N (2026) AI agent pull requests on GitHub: Frequency, structure, and merge conflict rates. arXiv preprint arXiv:260704697 https://doi.org/10.48550/arXiv.2607.04697	960 961 962
Yoshioka H, Monno T, Tokumasu H, et al (2026) Let’s make every pull request meaningful: An empirical analysis of developer and agentic pull requests. In: <i>Proceedings of the 23rd International Conference on Mining Software Repositories</i> , pp 747–751,	963 964 965 966

967 <https://doi.org/10.1145/3793302.3793567>
 968 Zhang P (2026) RepoReviewer: A local-first multi-agent architecture for repository-
 969 level code review. arXiv preprint arXiv:260316107 [https://doi.org/10.48550/arXiv](https://doi.org/10.48550/arXiv.2603.16107)
 970 [.2603.16107](https://doi.org/10.48550/arXiv.2603.16107), URL <https://arxiv.org/abs/2603.16107>
 971 Zhang X, Yu Y, Wang T, et al (2022) Pull request latency explained: an empirical
 972 overview. Empirical Software Engineering 27(6):126. [https://doi.org/10.1007/s106](https://doi.org/10.1007/s10664-022-10143-4)
 973 [64-022-10143-4](https://doi.org/10.1007/s10664-022-10143-4)
 974 Zhong S, Noei S, Adams B, et al (2026a) From human-centric to agentic code review:
 975 The impact of different generations of generative AI technology on review quality.
 976 arXiv preprint arXiv:260713196 <https://doi.org/10.48550/arXiv.2607.13196>
 977 Zhong S, Noei S, Zou Y, et al (2026b) Human-AI synergy in agentic code review.
 978 arXiv preprint arXiv:260315911
 979
 980
 981
 982
 983
 984
 985
 986
 987
 988
 989
 990
 991
 992
 993
 994
 995
 996
 997
 998
 999
 1000
 1001
 1002
 1003
 1004
 1005
 1006
 1007
 1008
 1009
 1010
 1011
 1012